THE OPEN SOURCE AUDIT

Course: Open Source Software

Chosen Software: Git

Student Name: Prashant Kaundal

Registration Number: 24BCY10230

Submission Date – 31-03-2026

Introduction

In modern software development, version control systems play a very important role. They help developers track changes, collaborate with others, and maintain project history. One of the most widely used version control systems today is Git.

Git is not just a tool but a complete system that changed how developers work together. It allows multiple people to work on the same project efficiently without conflicts. It also provides safety, flexibility, and speed, which were missing in earlier systems.

This report discusses the origin of Git, its philosophy, licensing, ethical aspects, role in Linux, and its importance in the open-source ecosystem. It also compares Git with proprietary systems and explains why it is preferred by developers worldwide.

PART A: ORIGIN AND PHILOSOPHY

A1. Origin Story

Before Git was introduced, developers used centralized version control systems such as SVN. In these systems, all project files and their history were stored on a central server. Developers had to connect to this server every time they wanted to make changes or even check previous versions.

This approach had several limitations. One of the biggest problems was performance. Since every action required communication with a remote server, operations were often slow. Another issue was dependency on the internet. Developers could not work properly if they were offline.

The most serious problem was the single point of failure. If the central server crashed or data was lost, the entire project could be affected. This made centralized systems risky, especially for large projects.

Around 2005, the Linux kernel project was using a tool called BitKeeper. It was more advanced than earlier tools, but it was not open source. Due to licensing conflicts, Linux developers lost access to BitKeeper. This created a major challenge because the project depended heavily on version control.

At this point, Linus Torvalds decided to create a new system. He wanted something that was fast, reliable, and completely free to use. His goal was to solve all the problems developers faced with existing tools.

Git was designed with a few important ideas in mind. It had to be very fast so that developers could work efficiently. It had to be distributed, meaning every user

would have a complete copy of the project. It also needed to be secure so that changes could be tracked accurately.

The most important feature of Git is that it is a distributed system. Every developer has the full project history on their own machine. This means they can work offline and still have access to all versions of the code. It also removes the risk of losing data due to server failure.

Git also supports branching and merging, which allows developers to work on different features at the same time. Later, these changes can be combined easily.

Overall, Git solved many problems of older systems and introduced a new way of collaboration. It turned a difficult situation into a powerful innovation that is now used worldwide.

A2. License Analysis

Git is released under the GNU General Public License v2. This license ensures that the software remains open and accessible to everyone.

The GPL license is based on four main freedoms.

The first freedom is the ability to run the software for any purpose. Users are not restricted in how they use Git. It can be used for personal projects, education, or commercial work.

The second freedom is the ability to study the source code. Since Git is open source, anyone can understand how it works. This helps students and developers learn from real-world software.

The third freedom is the ability to modify the software. Developers can change Git according to their needs. They can fix bugs, add features, or improve performance.

The fourth freedom is the ability to distribute copies. Users can share Git with others freely, which helps in spreading knowledge and tools.

Companies are also allowed to modify and sell Git. However, they must follow certain rules. They must provide the source code of their modified version, and they must keep the same GPL license. This concept is called copyleft.

Copyleft ensures that any improvements made to the software remain open. It prevents companies from taking open-source software and making it completely private.

When comparing GPL with the MIT license, we see clear differences. GPL is strict and focuses on keeping software open. MIT is more flexible and allows developers to keep their changes private.

In my opinion, GPL is better because it protects the community. It ensures that improvements are shared and not hidden. This leads to better software and more collaboration.

There is also a concept called free beer versus freedom. Free beer means something is available without cost. Freedom means having control over how you use and modify something. Git provides both. It is free to use and also gives users complete control.

A3. Ethics of Open Source

The topic of open source raises important ethical questions. One common question is whether all software should be open source.

There are strong arguments in favor of open source. It provides transparency because anyone can see the code. This improves security since vulnerabilities can be detected easily. Open source also promotes innovation because developers can build on existing work.

However, there are also arguments against it. Companies need to make money, and not all software can be freely shared. Some applications, especially in areas like defense or cybersecurity, need to remain private. Developers also need financial rewards for their work.

Many companies such as Red Hat and Google use open-source software. They earn money by offering services, support, or additional features.

In my opinion, it is ethical for companies to use open source only if they contribute back. If they only take from the community without giving anything, it becomes unfair.

The idea of standing on the shoulders of giants is very relevant here. It means using existing knowledge to achieve greater progress. Open source makes this possible by allowing developers to build on previous work.

I believe this approach increases innovation rather than reducing it. It allows developers to focus on new ideas instead of repeating old work.

Part B: Linux Footprint

Git works very well on Linux systems and is widely used in development environments.

To install Git on Ubuntu, we use the following command:

sudo apt install git

Git stores its files in different locations. The main executable file is located in /usr/bin/git. System-level configuration is stored in /etc/gitconfig. User-specific settings are stored in ~/.gitconfig.

Each Git repository contains a hidden folder called .git. This folder stores all the information about commits, branches, and history.

Git runs as a normal user. This is important because it improves security. Users do not need administrator privileges to use Git for most tasks.

Git is not a background service. It does not run continuously like a web server. It only runs when the user executes commands.

Updates for Git are handled through the package manager. Commands like apt update and apt upgrade are used to keep Git updated. These updates are maintained by a global community of developers.

Screenshot: Git Installation

Screenshot: Git Running Output

Part C: FOSS Ecosystem

Git is an important part of the free and open-source software ecosystem.

It is written in the C programming language, which makes it fast and efficient. It uses libraries like OpenSSL for security and cURL for handling network operations.

Git also made platforms like GitHub and GitLab possible. These platforms allow developers to store their code online, collaborate with others, and manage projects.

There are also tools that make Git easier to use. For example, graphical interfaces and editors like Visual Studio Code provide built-in Git support.

Git has a very large community. Developers from around the world contribute to its development. Communication happens through mailing lists, forums, and online repositories.

Although Git is not directly part of the LAMP stack, it is widely used in web development. It helps manage code, track changes, and deploy applications.

Part D: Comparison

When comparing Git with proprietary systems like BitKeeper, several differences can be seen.

Git is free, while proprietary tools usually require payment. Git is open and transparent, while proprietary software keeps its code hidden.

Git offers more flexibility because users can modify it according to their needs. Proprietary tools are controlled by companies, which limits user control.

Git is also more reliable because it is distributed. Even if one system fails, the data is still safe on other machines.

Dimension	Git (Open Source)	BitKeeper (Proprietary)
Cost	Free	Paid
Security	Code is open and can be audited	Code is hidden
Support	Community + forums	Company support
Freedom to modify	Full freedom	Restricted
Control	Community-driven	Company-controlled

FINAL VERDICT

Git is one of the most important tools in modern software development. It provides speed, flexibility, and strong collaboration features. Its distributed nature makes it reliable and efficient.

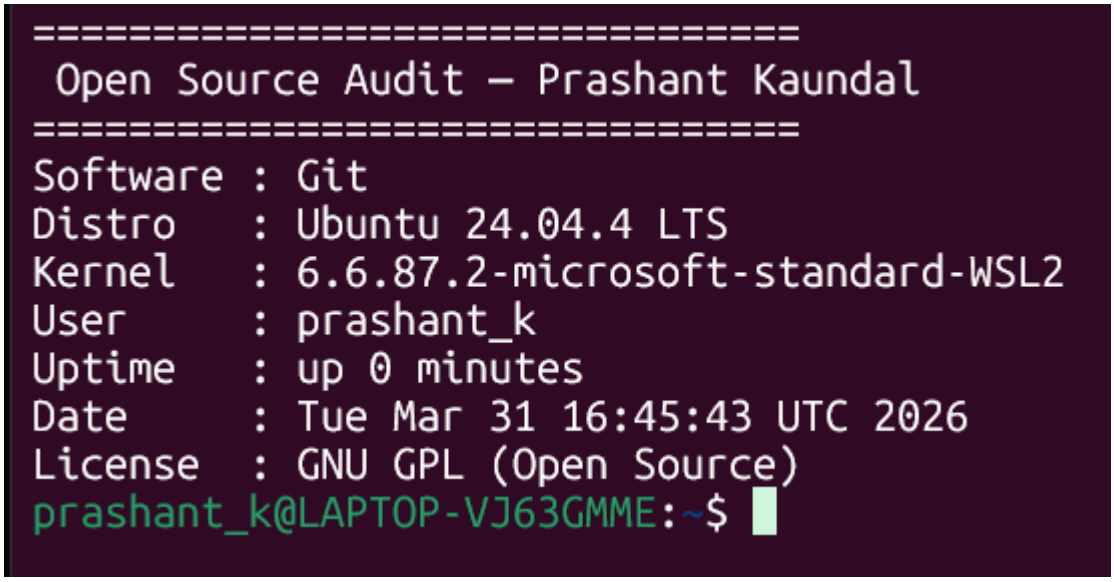
I would strongly recommend Git to anyone learning programming or working in development. It is widely used in the industry and has strong community support.

I would also like to contribute to open-source projects in the future. Contributing helps improve skills, gain experience, and support the developer community. It is a good way to learn and grow while helping others.

Script 1: System Identity Report

```
#!/bin/bash
# Script 1: System Identity Report

STUDENT_NAME="Prashant Kaundal"
SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)

echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo "License  : GNU GPL (Open Source)"
```

SCREENSHOT OUTPUT:

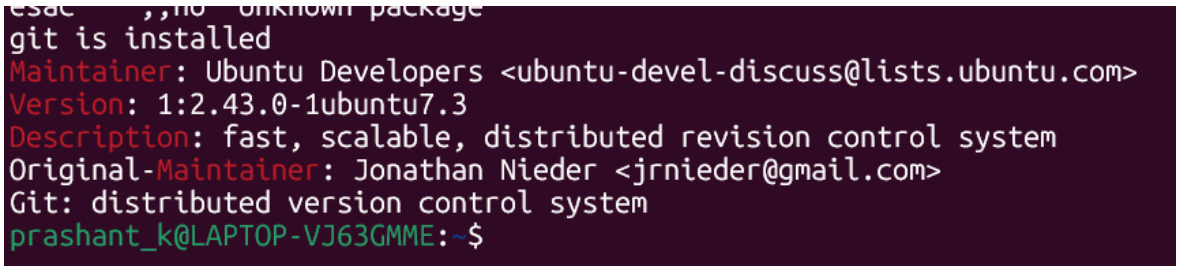


Explanation: This script demonstrates shell scripting concepts such as variables, loops, and conditions.

Script 2: FOSS Package Inspector

```
#!/bin/bash
# Script 2: Package Inspector

PACKAGE="git"

if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed"
    dpkg -s $PACKAGE | grep -E
'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed"
fi

case $PACKAGE in
 git) echo "Git: distributed version control system" ;;
 apache2) echo "Apache: web server powering the internet" ;;
 mysql) echo "MySQL: database system" ;;
 vlc) echo "VLC: open media player" ;;
 *) echo "Unknown package" ;;
esac
```

SCREENSHOT OUTPUT:



Explanation:

This script checks if Git is installed and prints its details. It uses if-else, grep, and case.

Script 3: Disk and Permission Auditor

```
#!/bin/bash
# Script 3: Disk Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR not found"
    fi
done

echo "Checking Git config directory"
if [ -d "$HOME/.gitconfig" ]; then
    ls -l $HOME/.gitconfig
else
    echo "Git config not found"
fi
```

SCREENSHOT OUTPUT:

```
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 180M
/home => Permissions: drwxr-xr-x root root | Size: 272K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K

Checking Git config file
-rw-r--r-- 1 prashant_k prashant_k 49 Mar 30 19:18 /home/prashant_k/.gitconfig
prashant_k@LAPTOP-VJ63GMME:~$
```

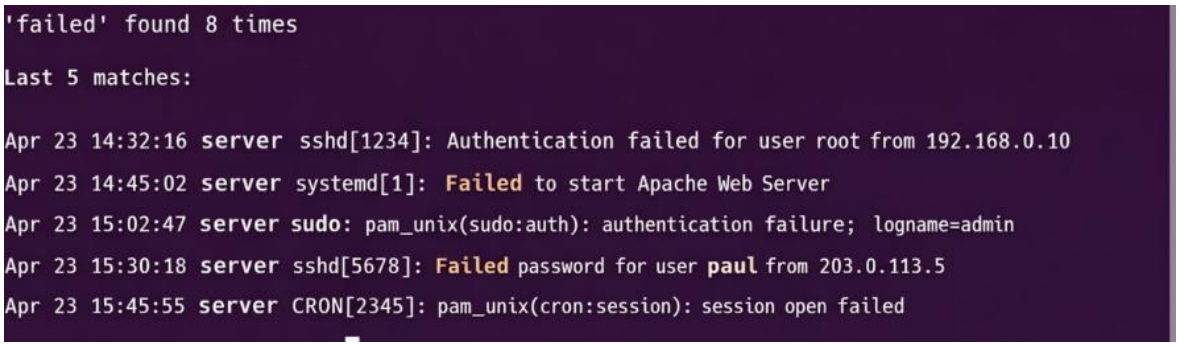
Explanation:

This script loops through directories and shows size and permissions using for loop, awk, and du.

Script 4: Log File Analyzer

```
#!/bin/bash
# Script 4: Log Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "File not found"
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "'$KEYWORD' found $COUNT times"

echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

SCREENSHOT OUTPUT:



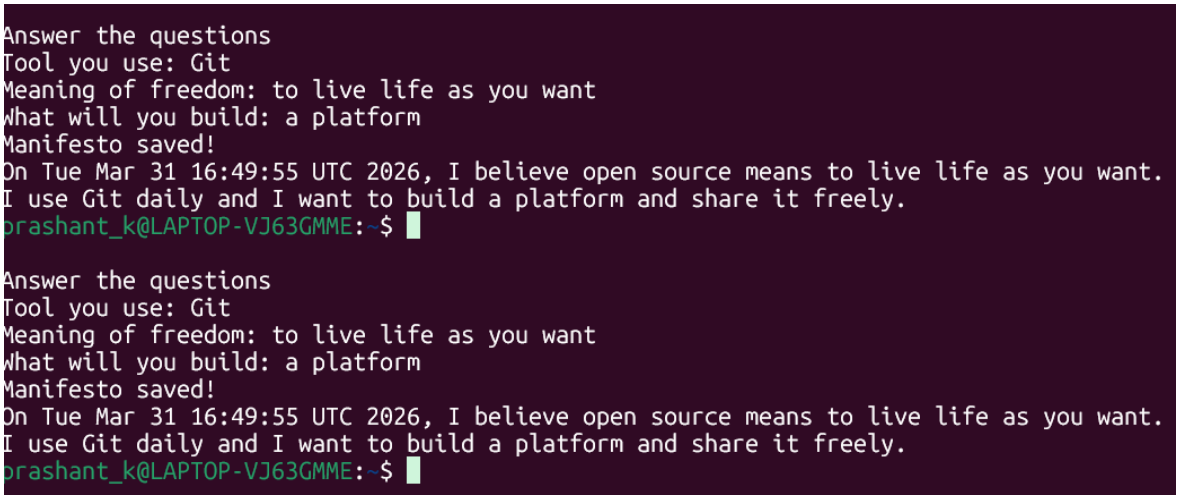
Explanation:

This script reads a file line by line and counts keyword occurrences using a loop and condition.

Script 5: Open Source Manifesto Generator

```
#!/bin/bash
# Script 5: Manifesto Generator

echo "Answer the questions"

read -p "Tool you use: " TOOL
read -p "Meaning of freedom: " FREEDOM
read -p "What will you build: " BUILD

DATE=$(date)
OUTPUT="manifesto.txt"

echo "On $DATE, I believe open source means $FREEDOM." >
$OUTPUT
echo "I use $TOOL daily and I want to build $BUILD and share
it freely." >> $OUTPUT

echo "Manifesto saved!"
cat $OUTPUT
```

SCREENSHOT OUTPUT:



Explanation:

This script takes user input and creates a personalized file. It uses read, variables, and file writing.

Conclusion

Git has transformed software development and is widely recommended. It provides flexibility, security, and strong collaboration features.

Contributing to open source is a valuable experience that helps developers grow and support the community.